DSA01- Object Oriented Programming With C++

Name: Kanish Kumar V **Register Number:** 192524361

Q:Scenario — Namma Metro Smart-Card Fare System

A city metro rail divides its network into three zones – Inner, Middle and Outer. The fare is Rs 10, Rs
20 or Rs 30 according to the farthest zone travelled, with a 20% surcharge during peak hours (8–10
AM and 5–8 PM). Passengers use a stored-value smart card, and a journey is allowed only if at least
Rs 50 remains on the card after the fare is deducted. Students and senior citizens receive a concession.

The first version of the gate software was written in a hurry. Some fares are off by a paisa, the peak
check accepts 10 AM as peak and 8 PM as off-peak inconsistently, a negative balance once appeared
on a card, and the fare values are typed in several places.

The number of journeys processed may grow:

1 station (20,000 journeys a day) → 1 line (500,000 a day) → whole network (3,000,000 a day)

You are required to analyse the problem, identify its objects, choose suitable C++ data types, constants, operators and control structures, implement the fare logic and validate it, so that every fare and balance is exact.

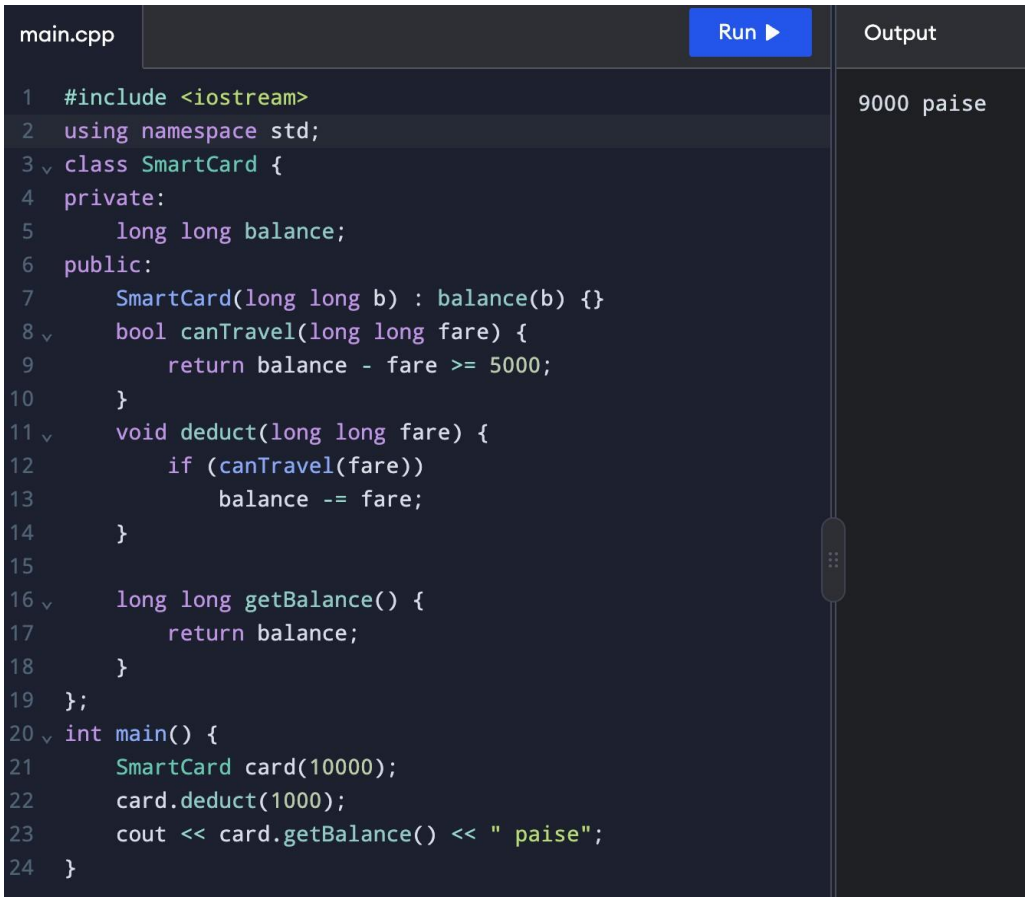
Design Challenge:

The metro is adding a fourth zone, a weekend discount and monthly passes.

Re-evaluate your original design and determine whether it should be retained, modified or Redesigned.

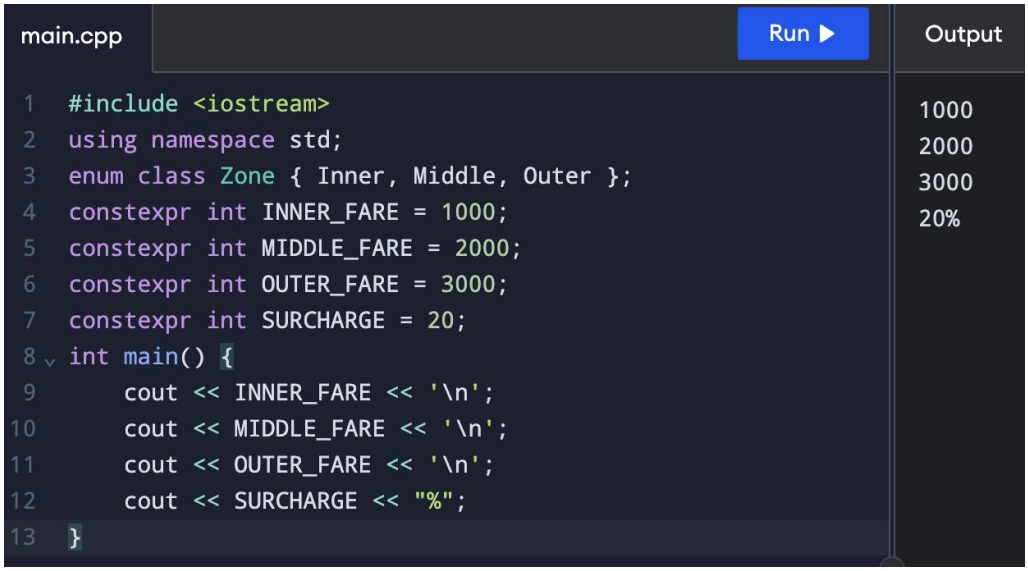
Task 1 – Identify the Objects

The main objects are SmartCard, Passenger, Journey, Zone and FareTable. Encapsulation protects the balance, abstraction hides calculations, inheritance supports passenger types, and polymorphism allows different concessions. The remaining balance must never be below ₹50 after a journey.



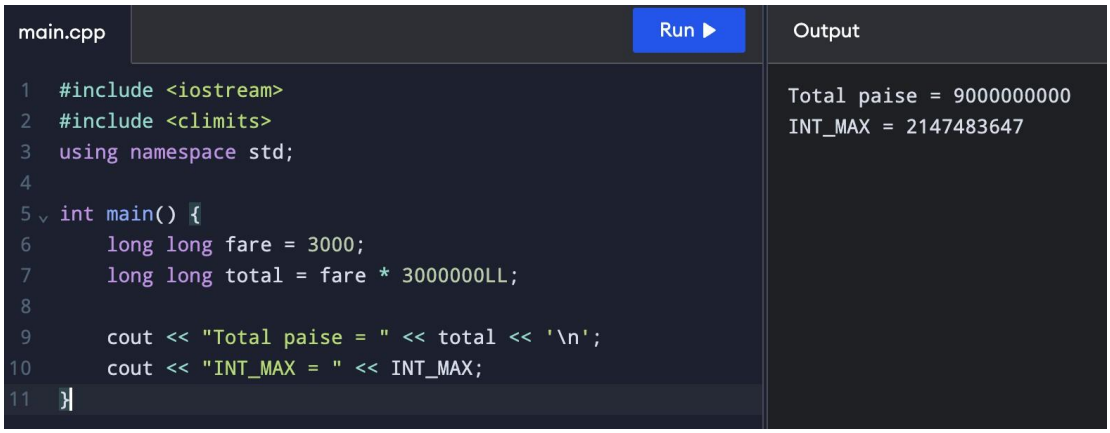
Task 2 – Tokens and Constants

Matter: Tokens are keywords, identifiers, constants, operators and punctuators. constexpr is preferred for fixed fares because it provides type checking and compile-time evaluation. enum class safely represents zones.



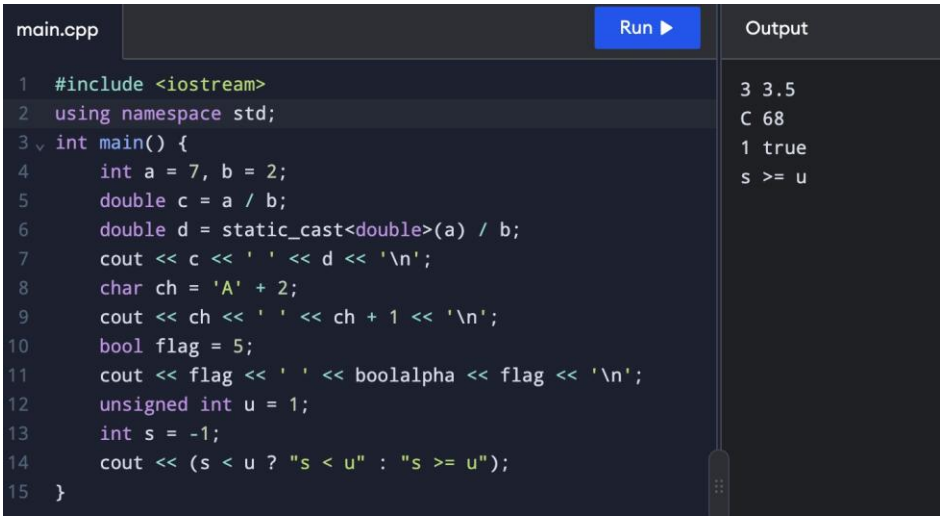
Task 3 – Data Types and Compatibility

Use long long for money in paise and journey totals, int for hours, and bool for peak status. Integer paise avoids floating-point rounding errors. A normal int may overflow when storing the total fare of three million journeys.



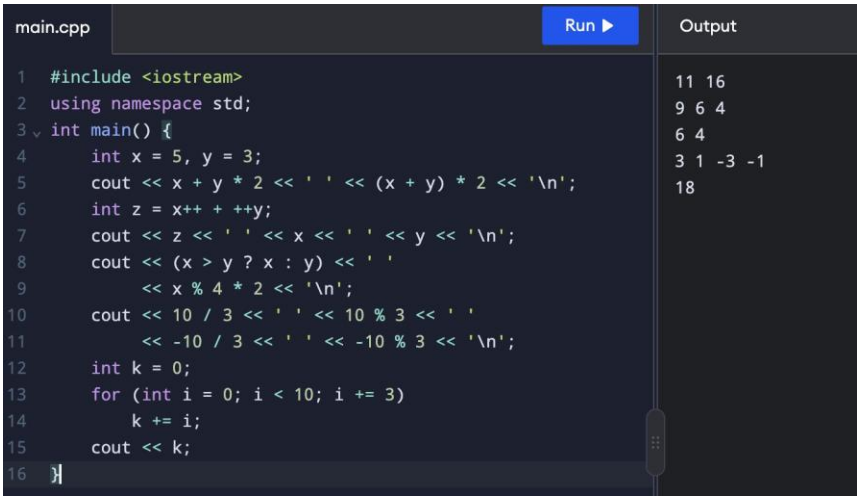
Task 4 – Implicit Conversions

Implicit conversion occurs automatically between data types. Integer division truncates decimals. static_cast performs explicit conversion, while brace initialization prevents narrowing conversions.



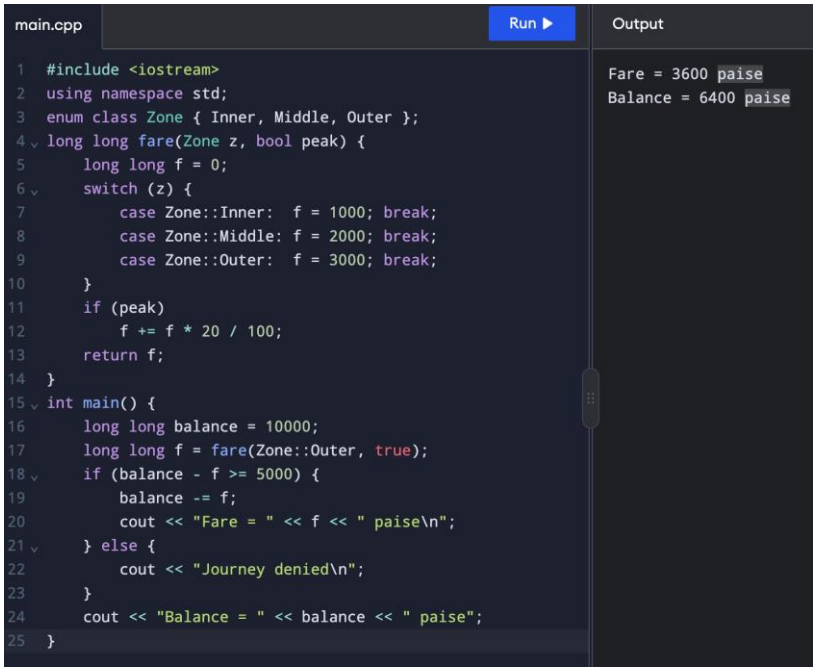
Task 5 – Operators and Precedence

Operator precedence determines the order of evaluation. Multiplication has higher precedence than addition. x++ increments after use, while ++x increments before use. Parentheses improve readability.



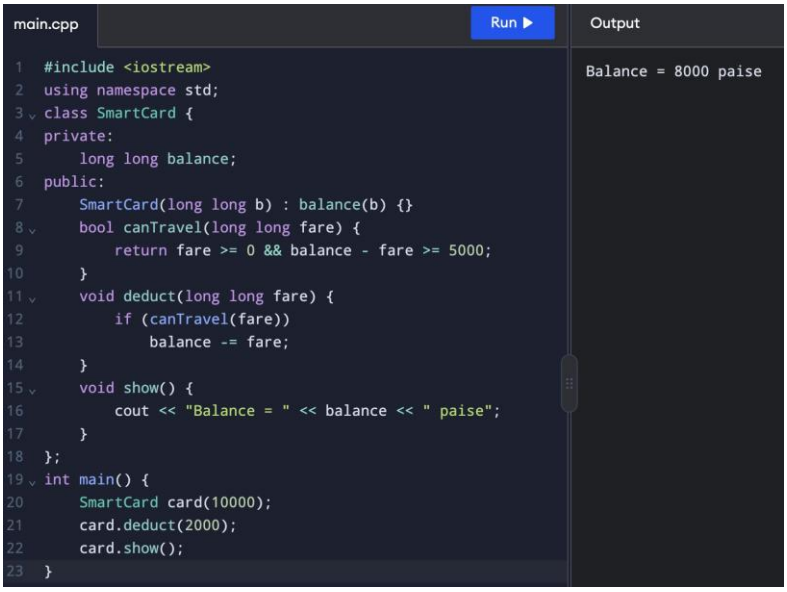
Task 6 – Control Structures

if-else checks conditions, switch selects a zone fare, and loops process multiple journeys. The peak surcharge is 20%. A journey is allowed only when at least ₹50 remains.



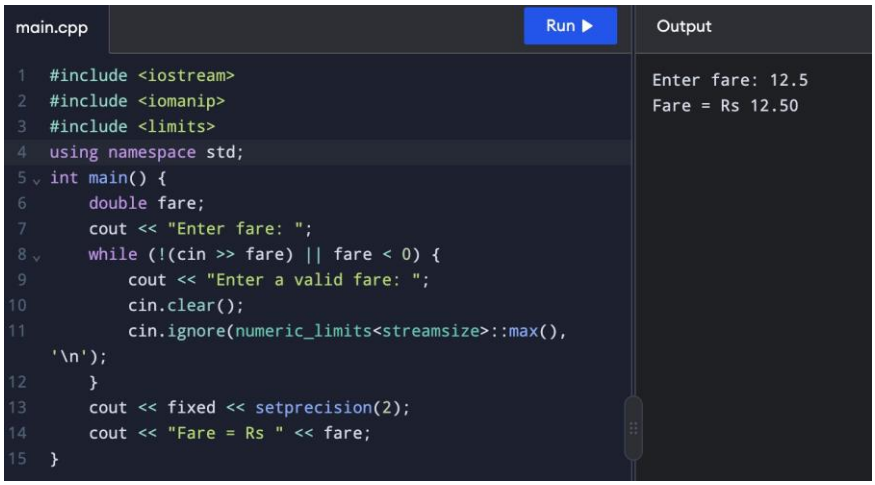
Task 7 – Procedural vs OOP

Procedural programming uses separate functions and variables. OOP combines data and functions in classes. A private balance prevents unauthorized changes and helps maintain the ₹50 minimum balance.



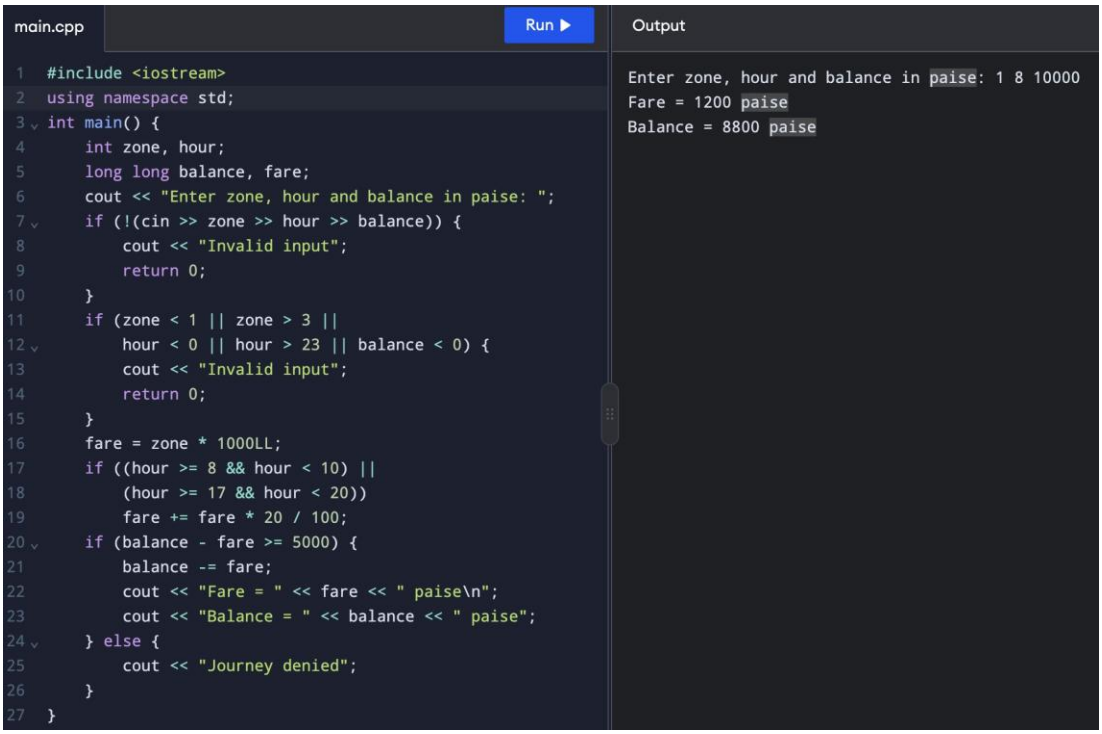
Task 8 – Safe Input and Output

cin reads input. cin.clear() resets input errors, and cin.ignore() removes invalid input. fixed and setprecision() display money neatly.



Task 9 – Implement and Test Fare Logic

The fare system must be tested with normal journeys, peak-hour boundaries, invalid zones and insufficient balances. Integer paise ensures exact fare calculations.



Task 10 – Compare Expected and Actual Results

int may overflow when storing large network totals. long long stores larger integer values, while double can introduce rounding errors. Therefore, integer paise with long long is recommended.



Task 11 – Final Design Recommendation

Use long long for money, constexpr for fixed fares, enum class for zones, and a SmartCard class for balance management. This design improves safety, readability and maintainability. Fare changes can be made in one place.

main.cpp Run ▶

```cpp
#include <iostream>
using namespace std;
constexpr long long FARES[] = {1000, 2000, 3000};
constexpr long long MIN_BALANCE = 5000;
class SmartCard {
private:
    long long balance;
public:
    SmartCard(long long b) : balance(b) {}
    bool pay(int zone) {
        if (zone < 1 || zone > 3)
            return false;
        long long fare = FARES[zone - 1];
        if (balance - fare < MIN_BALANCE)
            return false;
        balance -= fare;
        return true;
    }
  void show() {
        cout << "Balance = " << balance << " paise";
    }
};
int main() {
    SmartCard card(10000);
    if (card.pay(2))
        cout << "Payment successful\n";
    else
        cout << "Payment denied\n";
    card.show();
}
```

Output

```
Payment successful
Balance = 8000 paise
```

Task 12 – Design Challenge

The new requirements add an Airport Zone costing ₹60, a 10% weekend discount, and monthly passes for two zones. Modify the fare table, add a `Pass` class, and ensure the weekend discount does not combine with the peak surcharge.

main.cpp

Run

```
class FareCalculator {
private:
    // Assumed original fares and a new Airport fare.
    const array<double, 4> fares = {20, 30, 40, 60};

public:
    double calculateFare(Zone destination,
                         bool weekend,
                         bool peak) const {
        double fare = fares[static_cast<int>(destination)];

        // Weekend discount and peak surcharge do not combine.
        if (weekend) {
            fare = fare * 0.90;
        }
        else if (peak) {
            fare = fare * 1.20;  // Assumed 20% surcharge
        }

        return fare;
    }
};

class SmartCard {
private:
    double balance;
    FareCalculator calculator;
```

Output

```
===== METRO FARE MANAGEMENT =====
Fare table:
Zone 1  : Rs. 20
Zone 2  : Rs. 30
Zone 3  : Rs. 40
Airport : Rs. 60
Weekend discount: 10%
Peak surcharge: 20% (assumption)
Card balance: Rs. 200.00

TEST 1: Normal journey

Journey: 2 -> 1
Fare: Rs. 20.00
Journey successful!

TEST 2: Airport journey

Journey: 1 -> 4
Fare: Rs. 60.00
Journey successful!

TEST 3: Weekend journey

Journey: 1 -> 2
Fare: Rs. 27.00
Journey successful!
```